

Manejo de errores

Python para ciencias de la atmósfera

Lucía Pedraza Díaz

Tipos de errores

¿Qué tipo de error nos da?

```
print(float('hola'))
```

Tipos de errores

¿Qué tipo de error nos da?

```
print(float('hola'))
```



```
Traceback (most recent call last):  
  File "/Users/lu/Desktop/manejo_de_errores.py", line 3, in <module>  
    print(float('hola'))  
ValueError: could not convert string to float: 'hola'
```

Tipos de errores

¿Qué tipo de error nos da?

```
print(float('hola'))
```



```
print(9/0)
```

```
Traceback (most recent call last):  
  File "/Users/lu/Desktop/manejo_de_errores.py", line 3, in <module>  
    print(float('hola'))  
ValueError: could not convert string to float: 'hola'
```

Tipos de errores

¿Qué tipo de error nos da?

```
print(float('hola'))
```



```
Traceback (most recent call last):  
  File "/Users/lu/Desktop/manejo_de_errores.py", line 3, in <module>  
    print(float('hola'))  
ValueError: could not convert string to float: 'hola'
```

```
print(9/0)
```



```
print(9/0)  
ZeroDivisionError: division by zero
```

Tipos de errores

¿Qué tipo de error nos da?

```
print(float('hola'))
```



```
Traceback (most recent call last):  
  File "/Users/lu/Desktop/manejo_de_errores.py", line 3, in <module>  
    print(float('hola'))  
ValueError: could not convert string to float: 'hola'
```

```
print(9/0)
```



```
print(9/0)  
ZeroDivisionError: division by zero
```

```
l = [1,2,3]  
print(l[3])
```

Tipos de errores

¿Qué tipo de error nos da?

```
print(float('hola'))
```



```
Traceback (most recent call last):  
  File "/Users/lu/Desktop/manejo_de_errores.py", line 3, in <module>  
    print(float('hola'))  
ValueError: could not convert string to float: 'hola'
```

```
print(9/0)
```



```
print(9/0)  
ZeroDivisionError: division by zero
```

```
l = [1,2,3]  
print(l[3])
```



```
print(l[3])  
IndexError: list index out of range
```

Tipos de errores

¿Qué tipo de error nos da?

```
print(float('hola'))
```



```
Traceback (most recent call last):  
  File "/Users/lu/Desktop/manejo_de_errores.py", line 3, in <module>  
    print(float('hola'))  
ValueError: could not convert string to float: 'hola'
```

```
print(9/0)
```



```
print(9/0)  
ZeroDivisionError: division by zero
```

```
l = [1,2,3]  
print(l[3])
```



```
print(l[3])  
IndexError: list index out of range
```

```
d = {0: 120}  
print(d[9])
```

Tipos de errores

¿Qué tipo de error nos da?

```
print(float('hola'))
```

```
Traceback (most recent call last):  
  File "/Users/lu/Desktop/manejo_de_errores.py", line 3, in <module>  
    print(float('hola'))  
ValueError: could not convert string to float: 'hola'
```

```
print(9/0)
```

```
print(9/0)  
ZeroDivisionError: division by zero
```

```
l = [1,2,3]  
print(l[3])
```

```
print(l[3])  
IndexError: list index out of range
```

```
d = {0: 120}  
print(d[9])
```

```
print(d[9])  
KeyError: 9
```

Tipos de errores

¿Qué tipo de error nos da?

```
print(float('hola'))
```

```
Traceback (most recent call last):  
  File "/Users/lu/Desktop/manejo_de_errores.py", line 3, in <module>  
    print(float('hola'))  
ValueError: could not convert string to float: 'hola'
```

```
print(9/0)
```

```
print(9/0)  
ZeroDivisionError: division by zero
```

```
l = [1,2,3]  
print(l[3])
```

```
print(l[3])  
IndexError: list index out of range
```

```
d = {0: 120}  
print(d[9])
```

```
print(d[9])  
KeyError: 9
```

```
print(5 + 'hola')
```

Tipos de errores

¿Qué tipo de error nos da?

```
print(float('hola'))
```

```
Traceback (most recent call last):
  File "/Users/lu/Desktop/manejo_de_errores.py", line 3, in <module>
    print(float('hola'))
ValueError: could not convert string to float: 'hola'
```

```
print(9/0)
```

```
print(9/0)
ZeroDivisionError: division by zero
```

```
l = [1,2,3]
print(l[3])
```

```
print(l[3])
IndexError: list index out of range
```

```
d = {0: 120}
print(d[9])
```

```
print(d[9])
KeyError: 9
```

```
print(5 + 'hola')
```

```
print(5 + 'hola')
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

```
if 4:
    print('hola')
```

Tipos de errores

¿Qué tipo de error nos da?

```
print(float('hola'))
```

```
Traceback (most recent call last):
  File "/Users/lu/Desktop/manejo_de_errores.py", line 3, in <module>
    print(float('hola'))
ValueError: could not convert string to float: 'hola'
```

```
print(9/0)
```

```
print(9/0)
ZeroDivisionError: division by zero
```

```
l = [1,2,3]
print(l[3])
```

```
print(l[3])
IndexError: list index out of range
```

```
d = {0: 120}
print(d[9])
```

```
print(d[9])
KeyError: 9
```

```
print(5 + 'hola')
```

```
print(5 + 'hola')
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

```
if 4:
    print('hola')
```

```
print('hola')
^
IndentationError: expected an indented block
```

Estructura try/except

Sirve para manejar errores en tiempo de ejecución sin que el programa se caiga.

La idea: pones en el bloque try el código que podría fallar, y en except lo que quieres que pase si falla.

try:

Bloque de código que
puede fallar

except:

Lo que pasa si falla el try

Ejercicio

```
temps = ['20', '18.7', '15.2', '8', 'N/A', '8', '34', '-999']  
  
# ejercicio: calcular la temperatura promedio
```

Ejercicio

not available

Indica que falta un dato

```
temps = ['20', '18.7', '15.2', '8', 'N/A', '8', '34', '-999']
```

```
# ejercicio: calcular la temperatura promedio
```

```
temps_float = []
for temp in temps:
    temps_float.append(float(temp))

promedio = sum(temps_float)/len(temps_float)
print(f'la temperatura promedio es {promedio:.2f} celcius')
```

Ejercicio

not available

Indica que falta un dato

```
temps = ['20', '18.7', '15.2', '8', 'N/A', '8', '34', '-999']  
  
# ejercicio: calcular la temperatura promedio
```

```
temps_float = []  
for temp in temps:  
    temps_float.append(float(temp))  
  
promedio = sum(temps_float)/len(temps_float)  
print(f'la temperatura promedio es {promedio:.2f} celcius')
```

```
    temps_float.append(float(temp))  
ValueError: could not convert string to float: 'N/A'
```

Ejercicio

not available

Indica que falta un dato

```
temps = ['20', '18.7', '15.2', '8', 'N/A', '8', '34', '-999']  
  
# ejercicio: calcular la temperatura promedio
```

```
temps_float = []  
for temp in temps:  
    temps_float.append(float(temp))  
  
promedio = sum(temps_float)/len(temps_float)  
print(f'la temperatura promedio es {promedio:.2f} celcius')
```

```
    temps_float.append(float(temp))  
ValueError: could not convert string to float: 'N/A'
```

Queremos manejar los errores por elemento (por iteración)

Ejercicio

```
temps_float = []
for temp in temps:
    try:
        temps_float.append(float(temp))
    except:
        pass

promedio = sum(temps_float)/len(temps_float)
print(f'la temperatura promedio es {promedio:.2f} celcius')
```

Ejercicio

```
temps_float = []
for temp in temps:
    try:
        temps_float.append(float(temp))
    except:
        pass

promedio = sum(temps_float)/len(temps_float)
print(f'la temperatura promedio es {promedio:.2f} celcius')
```

```
temps_float = []
for temp in temps:
    try:
        temps_float.append(float(temp))
    except:
        print(f'Valor no válido :{temp}. La temperatura debe ser flotante')

promedio = sum(temps_float)/len(temps_float)
print(f'la temperatura promedio es {promedio:.2f} celcius')
```

Ejercicio

```
temps_float = []
for temp in temps:
    try:
        if float(temp) == -999:
            continue
        temps_float.append(float(temp))
    except:
        print(f'Valor no válido :{temp}. La temperatura debe ser flotante')

promedio = sum(temps_float)/len(temps_float)
print(f'la temperatura promedio es {promedio:.2f} celcius')
```

Ejercicio

```
temps_float = []
for temp in temps:
    try:
        T = float(temp)
        if T == -999:
            pass
        else:
            temps_float.append(T)

        print(9/0)
    except:
        print(f'Valor no válido :{temp}. La temperatura debe ser flotante')

promedio = sum(temps_float)/len(temps_float)
print(f'la temperatura promedio es {promedio:.2f} celcius')
```

print(9/0)
ZeroDivisionError: division by zero

Ejercicio

```
temps_float = []
for temp in temps:
    try:
        T = float(temp)
        if T == -999:
            pass
        else:
            temps_float.append(T)

        print(9/0)
    except ValueError:
        print(f'Valor no válido :{temp}. La temperatura debe ser flotante')

promedio = sum(temps_float)/len(temps_float)
print(f'la temperatura promedio es {promedio:.2f} celcius')
```

Especificamos el tipo de error

Ejercicio

```
temps_float = []
for temp in temps:
    try:
        T = float(temp)
        if T == -999:
            pass
        else:
            temps_float.append(T)

        print(9/0)
    except ValueError:
        print(f'Valor no válido :{temp}. La temperatura debe ser flotante')
    except ZeroDivisionError:
        print('CUIDADO. DIVISION POR CERO')

promedio = sum(temps_float)/len(temps_float)
print(f'la temperatura promedio es {promedio:.2f} celcius')
```

Ejercicio

```
def a_flotante(x):  
    try:  
        return float(x)  
    except:  
        return None  
  
temps_clean = []  
for temp in temps:  
    T = a_flotante(temp)  
  
    if T is not None:  
        temps_clean.append(T)  
  
print(temps_clean)
```

Ejercicio

```
def a_flotante(x):  
    try:  
        return float(x)  
    except:  
        return None  
  
temps_clean = []  
for temp in temps:  
    T = a_flotante(temp)  
  
    if T is not None:  
        temps_clean.append(T)  
  
print(temps_clean)
```

```
def a_flotante(x):  
    try:  
        return float(x)  
    except:  
        return None  
  
temps_clean = [a_flotante(temp) for temp in temps if a_flotante(temp) is not None]  
print(temps_clean)
```

Ejercicio

```
def a_flotante(x):  
    try:  
        return float(x)  
    except:  
        return None  
  
temps_clean = []  
for temp in temps:  
    T = a_flotante(temp)  
  
    if T is not None:  
        temps_clean.append(T)  
  
print(temps_clean)
```

```
def a_flotante(x):  
    try:  
        return float(x)  
    except:  
        return None  
  
temps_clean = [a_flotante(temp) for temp in temps if a_flotante(temp) is not None]  
print(temps_clean)
```

¿Cómo agregamos la condición para manejar el -999?

Ejercicio

```
def a_flotante(x):  
    try:  
        return float(x)  
    except:  
        return None  
  
temps_clean = [a_flotante(temp) for temp in temps if a_flotante(temp) is not None  
               and a_flotante(temp) != -999]  
print(temps_clean)
```

Ejercicio

Estamos llamando a la función 3 veces

```
def a_flotante(x):  
    try:  
        return float(x)  
    except:  
        return None  
  
temps_clean = [a_flotante(temp) for temp in temps if a_flotante(temp) is not None  
               and a_flotante(temp) != -999]  
print(temps_clean)
```

Ejercicio

Estamos llamando a la función 3 veces

```
def a_flotante(x):
    try:
        return float(x)
    except:
        return None

temps_clean = [a_flotante(temp) for temp in temps if a_flotante(temp) is not None
               and a_flotante(temp) != -999]
print(temps_clean)
```

```
def a_flotante(x):
    try:
        return float(x)
    except:
        return None

temps_clean = [T for temp in temps if (T:=a_flotante(temp)) is not None
               and T != -999]
print(temps_clean)
```

:=

Operador walrus

El walrus asigna un valor a una variable y a la vez devuelve ese valor, todo dentro de una sola expresión.

try/except/else/finally

```
numeros = ['20', '18.7', '15.2', '8', 'N/A', '8', '34', '-999']

exitosas = 0
fallidas = 0
revisados = 0

for num in numeros:
    try:
        n = float(num)
    except ValueError:
        fallidas += 1
    else:
        exitosas += 1
    finally:
        revisados += 1

print(f"Convertidas: {exitosas}, fallidas: {fallidas}, revisados en total: {revisados}")
```

raise

```
temps = ['20', '18.7', '15.2', '8', 'N/A', '8', '34', '-999']
# ejercicio: calcular la temperatura promedio

temps_float = []
for temp in temps:
    try:
        if float(temp) == -999:
            continue
        temps_float.append(float(temp))
    except:
        print(f'Valor no válido :{temp}. La temperatura debe ser flotante')

promedio = sum(temps_float)/len(temps_float)
print(f'la temperatura promedio es {promedio:.2f} celcius')
```

raise

```
temps = ['20', '18.7', '15.2', '8', 'N/A', '8', '34', '-999']
# ejercicio: calcular la temperatura promedio

temps_float = []
for temp in temps:
    try:
        if float(temp) == -999:
            raise ValueError ('-999 no es una temperatura válida')
        temps_float.append(float(temp))
    except:
        print(f'Valor no válido :{temp}')

promedio = sum(temps_float)/len(temps_float)
print(f'la temperatura promedio es {promedio:.2f} celcius')
```

raise

```
temps = ['20', '18.7', '15.2', '8', 'N/A', '8', '34', '-999']
# ejercicio: calcular la temperatura promedio

temps_float = []
for temp in temps:
    try:
        if float(temp) == -999:
            raise ValueError('-999 no es una temperatura válida')
        temps_float.append(float(temp))
    except ValueError as e:
        print(f'Valor no válido :{temp}')
        print(e)

promedio = sum(temps_float)/len(temps_float)
print(f'la temperatura promedio es {promedio:.2f} celcius')
```

assert

- Verifica que una condición sea verdadera. Si lo es, no pasa nada y el programa sigue. Si es falsa, lanza AssertionError y detiene el programa.

assert condicion, "mensaje si falla"

```
temps = [10, 20, 23]

validos = []
for t in temps:
    try:
        validos.append(float(t))
    except ValueError:
        pass

promedio = sum(validos) / len(validos)
print(promedio)
```

¿Qué pasa si
cambiamos los tres
valores por 'N/A'?

assert

- Verifica que una condición sea verdadera. Si lo es, no pasa nada y el programa sigue. Si es falsa, lanza AssertionError y detiene el programa.

assert condicion, "mensaje si falla"

```
temps = [10, 20, 23]

validos = []
for t in temps:
    try:
        validos.append(float(t))
    except ValueError:
        pass

promedio = sum(validos) / len(validos)
print(promedio)
```

```
temps = ['N/A', 'N/A', 'N/A']

validos = []
for t in temps:
    try:
        validos.append(float(t))
    except ValueError:
        pass

assert len(validos) > 0, 'ningún dato válido'
promedio = sum(validos) / len(validos)
print(promedio)
```